TÀI LIỆU BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

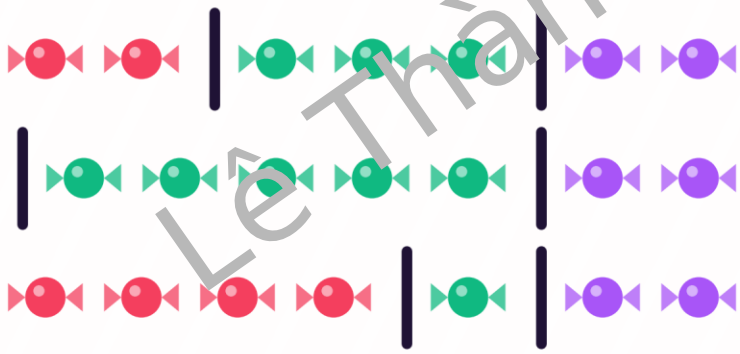
BÀI TOÁN
CHIA KẸO EULER

TẬP 1 · LÝ THUYẾT

Kẹo và vách ngăn · Tổ hợp lặp · Hàm sinh

Lê Thành Nam

Chia 7 viên kẹo cho 3 bạn



(2, 3, 2)

(0, 5, 2)

(4, 1, 2)

$$C(n+k-1, k-1) = C(9, 2) = 36$$

01	02	03	04
Chuẩn bị	Chia kẹo	Có chặn	Mở rộng

12 bài học · 4 chương · lý thuyết đầy đủ trước, bài toán sau

Tổng hợp và biên soạn: Lê Thành Nam

LỜI NÓI ĐẦU

"Có bao nhiêu cách chia 10 viên kẹo giống nhau cho 3 bạn?" Câu hỏi nghe như đố vui ấy mở ra cả một thế giới **đếm**: nghiệm nguyên của phương trình, tổ hợp lặp, xếp ghế có khoảng cách, đường đi trên lưới, phân phối đồ vật vào hộp, giỏ trái cây... Tất cả bắt đầu từ **bài toán chia kẹo Euler** và một ý tưởng rất đẹp: biến mỗi cách chia thành một hàng gồm **kẹo và vách ngăn**, rồi chỉ cần chọn chỗ đặt vách. Đáp số: C(12, 2) = 66.

Ý tưởng ấy là một **song ánh**: thay vì đếm thứ khó đếm, ta tìm một tương ứng một-một với thứ dễ đếm. Cộng thêm vài công cụ (đổi biến cho chặn dưới, bao hàm – loại trừ cho chặn trên, quy hoạch động với tổng tiền tố, định lí Lucas khi lấy dư cho số nhỏ, nguyên lí phản xạ cho đường đi, hàm sinh để ghép mọi thứ), ta giải được hầu hết các bài đếm trong đề thi học sinh giỏi.

Bộ tài liệu gồm ba tập đi cùng nhau:

- **Tập 1: Lý thuyết** (cuốn sách này) gồm 12 bài học chia thành 4 chương: Kiến thức chuẩn bị; Chia kẹo Euler cơ bản; Chia kẹo có chặn; Mở rộng: đường đi, phân phối và hàm sinh.
- **Tập 2: Bài tập** gồm 36 bài, mỗi bài học có ba bài tập ở ba mức độ, có ràng buộc theo subtask như đề thi thật.
- **Tập 3: Hướng dẫn giải** phân tích từng bài tập từ cách ngây thơ đến cách tối ưu, kèm chương trình hoàn chỉnh và bảng điểm thực tế của từng cách làm, kể cả những cách **sai** hoặc **đúng nhưng chậm**.

Sách được viết cho người **bắt đầu từ con số 0**: chương 1 ôn lại từ đầu quy tắc đếm, tổ hợp, phần nguyên, số dư, lũy thừa nhanh, nghịch đảo modulo và bảng giai thừa, những công cụ dùng trong **mọi** bài của sách. Mỗi bài học trình bày **đầy đủ lý thuyết trước** (định nghĩa, định lí, chứng minh, ví dụ, hình vẽ), rồi mới vào **phần bài toán** để cài đặt. Mọi con số trong sách (thời gian chạy, kết quả chạy tay) đều là số đo và kết quả thật trên máy. Mọi chương trình đều đã được đối chiếu với lời giải vét cạn trên hàng trăm bộ dữ liệu ngẫu nhiên.

Bộ sách này là "người anh em" của bộ *Nguyên lí Dirichlet*: hai bộ dùng chung chương Kiến thức chuẩn bị và có thể học độc lập.

Chương trình được viết bằng C++ theo chuẩn C++14, đọc và ghi tệp theo kiểu "có tệp thì đọc tệp" để dùng được cả với phần mềm chấm Themis lẫn các trang chấm trực tuyến.

Chúc các em học tập say mê và đạt kết quả cao!

Lê Thành Nam

Lê Thành Nam

CÁCH HỌC VỚI TÀI LIỆU NÀY

Mỗi bài học gồm hai phần lớn:

Phần	Nội dung
Mục tiêu, Khởi động	Những gì em sẽ làm được sau bài học, và một tình huống đời thường dẫn tới ý tưởng chính.
PHẦN LÝ THUYẾT	Định nghĩa, định lí, chứng minh, ví dụ chạy tay, hình minh họa. Đọc phần này với giấy bút, như học một bài Toán.
PHẦN BÀI TOÁN	Một bài toán lập trình cụ thể dùng đúng lý thuyết vừa học: cách chậm (kèm số đo thời gian thật), cách nhanh, chương trình hoàn chỉnh, phân tích độ phức tạp.
Bẫy thường gặp	Những lỗi đã làm mất điểm của nhiều thế hệ học sinh.
Luyện tập	Liên hệ tới các bài tập ở Tập 2 và vài câu tự kiểm tra có đáp số.

Các khung màu trong sách có ý nghĩa cố định:

Khung	Ý nghĩa
KHỞI ĐỘNG	Câu hỏi mở đầu bài học.
ĐỊNH NGHĨA	Khái niệm mới, cần hiểu chính xác.
ĐỊNH LÍ	Kết quả quan trọng, luôn kèm chứng minh.
CHỨNG MINH	Lập luận vì sao định lí đúng.
VÍ DỤ	Áp dụng công thức trên số cụ thể.
CÔNG THỨC	Công thức cần nhớ.
Ý TƯỞNG	Hướng giải hoặc các bước của một phương pháp.
BÀI TOÁN	Đề bài lập trình của bài học.
ĐẾM SỐ PHÉP TÍNH	Phân tích độ phức tạp và số đo thời gian thật.
VÌ SAO ĐÚNG?	Giải thích thêm một bước khó.
LƯU Ý	Chi tiết dễ bỏ sót.
BẪY THƯỜNG GẶP	Các lỗi hay gặp.
GHI NHỚ	Tóm tắt cuối bài.

Trong các hình chia kẹo, **kẹo** là viên kẹo có hai đầu giấy gói, **vách ngăn** là vạch đậm màu tối. Ô **xanh lá** là kết quả hoặc số hạng mang dấu cộng, ô **đỏ** là phần bị trừ hoặc phần vi phạm.

Lời khuyên khi học: Với mỗi bài, hãy tự hỏi: **kẹo là gì, ai nhận kẹo, kẹo (và người nhận) giống hay khác nhau, mỗi người nhận ít nhất, nhiều nhất bao nhiêu?** Chỉ khi tự viết lại được chương trình mà không nhìn sách, em mới thật sự nắm được bài.

Lê Thành Nam

MỤC LỤC

LỜI NÓI ĐẦU **2**

CÁCH HỌC VỚI TÀI LIỆU NÀY **3**

CHƯƠNG 1. KIẾN THỨC CHUẨN BỊ **5**

Bài 1. Quy tắc đếm, hoán vị và tổ hợp 5
Bài 2. Phần nguyên và số dư 13
Bài 3. Lũy thừa nhanh, nghịch đảo và bảng giai thừa 19

CHƯƠNG 2. CHIA KẸO EULER CƠ BẢN **25**

Bài 4. Kẹo và vách ngăn 25
Bài 5. Chặn dưới và đổi biến 32
Bài 6. Tổ hợp lặp 39

CHƯƠNG 3. CHIA KẸO CÓ CHẶN **45**

Bài 7. Chặn trên và bao hàm – loại trừ 45
Bài 8. Quy hoạch động chia kẹo 51
Bài 9. Định lí Lucas và số dư Trung Hoa 56

CHƯƠNG 4. MỞ RỘNG: ĐƯỜNG ĐI, PHÂN PHỐI VÀ HÀM SINH **63**

Bài 10. Đường đi trên lưới và nguyên lí phản xạ 63
Bài 11. Phân phối đồ vật vào hộp 70
Bài 12. Nhiều loại kẹo và hàm sinh 76

PHỤ LỤC: BẢNG TRA CỨU NHANH **83**

Lê Thành Nam

CHƯƠNG 1

KIẾN THỨC CHUẨN BỊ

*Trước khi gặp bài toán chia kẹo Euler, em cần ba "dụng cụ" nhỏ. Thứ nhất là các **quy tắc đếm** (cộng, nhân) và ba khái niệm hoán vị, chỉnh hợp, tổ hợp, vì chia kẹo Euler cho đáp án dưới dạng một tổ hợp C(n, k). Thứ hai là **phần nguyên và số dư**: nhiều bài chia kẹo có điều kiện chẵn lẻ, chia hết, chia đều [n/k], và mọi đáp số lớn đều phải lấy dư. Thứ ba là cách tính **tổ hợp theo modulo** khi số rất lớn: lũy thừa nhanh, nghịch đảo modulo, bảng giai thừa. Đây là công cụ dùng trong **mọi** bài của sách. Chương này viết cho người bắt đầu từ con số 0: mọi khái niệm đều có ví dụ đếm tay, hình vẽ, và chương trình chạy thử.*

■ TRONG CHƯƠNG NÀY

- **Bài 1.** Quy tắc đếm, hoán vị và tổ hợp
- **Bài 2.** Phần nguyên và số dư
- **Bài 3.** Lũy thừa nhanh, nghịch đảo và bảng giai thừa

Lê Thành Nam

BÀI 1 Quy tắc đếm, hoán vị và tổ hợp

Học xong bài này, em sẽ:

- ✓ Phân biệt và dùng đúng **quy tắc cộng** (chọn "hoặc") và **quy tắc nhân** (làm "rồi").
- ✓ Hiểu ba khái niệm **hoán vị** n!, **chỉnh hợp** A(n, k), **tổ hợp** C(n, k) và vì sao C(n, k) = n! / (k!·(n − k)!).
- ✓ Lập **tam giác Pascal** bằng công thức C(n, k) = C(n − 1, k − 1) + C(n − 1, k) và hiểu vì sao nó đúng.
- ✓ Viết chương trình tính C(n, k) theo modulo, và thấy vì sao không thể dùng thẳng giai thừa hay đệ quy không nhớ.

■ KHỞI ĐỘNG

Căng tin trường có 3 món chính và 2 món tráng miệng. Nếu em chỉ được chọn **một** món thì có bao nhiêu cách? Nếu em chọn **một món chính và một món tráng miệng** thì sao? Lớp em có 40 bạn, cần cử 3 bạn đi trực nhật: có bao nhiêu cách? Ba câu hỏi rất đời thường này mở ra toàn bộ cuốn sách: bài toán chia kẹo Euler và mọi biến thể của nó đều xây trên những phép đếm như vậy.

PHẦN LÝ THUYẾT

1. Quy tắc cộng và quy tắc nhân

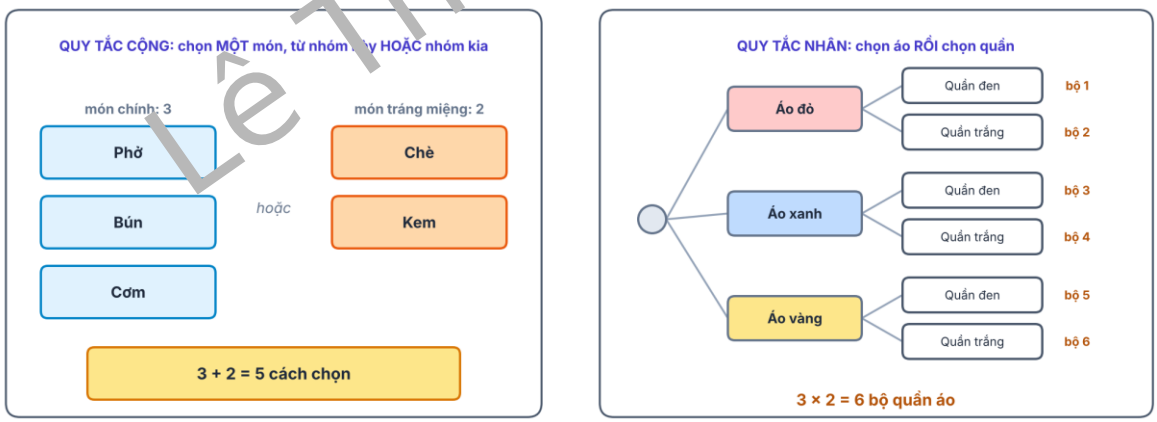
■ QUY TẮC CỘNG

Nếu một việc có thể làm theo **phương án A** (có m cách) **hoặc** theo **phương án B** (có n cách), và không có cách nào thuộc cả hai phương án, thì có **m + n** cách làm việc đó.

■ QUY TẮC NHÂN

Nếu một việc gồm **hai bước liên tiếp**: bước 1 có m cách, và **ứng với mỗi** cách của bước 1, bước 2 có n cách, thì có **m · n** cách làm việc đó.

Lê Thành Nam



Hình 1.1. Quy tắc cộng: chọn một món trong hai nhóm. Quy tắc nhân: mỗi nhánh áo tách ra hai nhánh quần

■ VÍ DỤ 1

Biển số xe gồm 2 chữ cái (A..Z, 26 chữ) rồi 3 chữ số (0..9). Có bao nhiêu biển số?

■ LỜI GIẢI

Năm bước liên tiếp: 26 · 26 · 10 · 10 · 10 = **676 000** biển số (quy tắc nhân áp dụng nhiều lần).

■ LƯU Ý

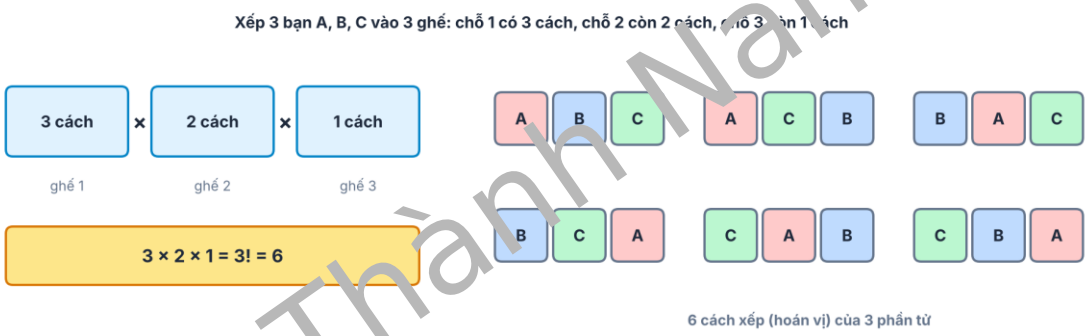
Dấu hiệu nhận biết: câu hỏi có chữ **"hoặc"** thì nghĩ tới phép **cộng**; có chữ **"rồi", "và", "lần lượt"** thì nghĩ tới phép **nhân**. Quy tắc cộng đòi hỏi các phương án **không trùng nhau**; khi có trùng, ta cần bao hàm – loại trừ.

2. Hoán vị: xếp n vật thành hàng

■ HOÁN VỊ

Mỗi cách sắp xếp n vật **khác nhau** thành một hàng gọi là một **hoán vị** của n vật. Số hoán vị là $n! = n \cdot (n-1) \cdot (n-2) \cdots 2 \cdot 1$ (đọc là "n giai thừa"), quy ước $0! = 1$.

Vì sao? Chỗ thứ nhất có n lựa chọn, chỗ thứ hai còn $n-1$ lựa chọn (đã dùng một vật), ..., chỗ cuối còn 1 lựa chọn. Theo quy tắc nhân ta được n!.



Hình 1.2. *Sáu hoán vị của A, B, C*

Giai thừa tăng **rất nhanh**: $10! = 3\,628\,800$, $20! \approx 2{,}4 \cdot 10^{18}$ (vừa đủ kiểu `long long`), 21! đã tràn số. Đây là lí do ta phải tính theo **modulo**.

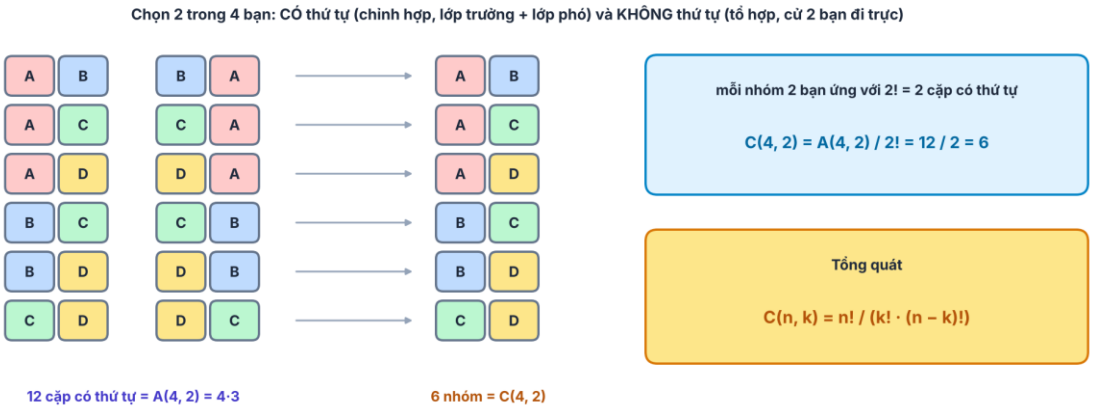
3. Chỉnh hợp và tổ hợp: chọn k vật trong n vật

■ CHỈNH HỢP VÀ TỔ HỢP

Chọn k vật trong n vật khác nhau.

- Nếu **thứ tự có ý nghĩa** (ví dụ chọn lớp trưởng rồi lớp phó): mỗi cách là một **chỉnh hợp**, có $A(n, k) = n \cdot (n-1) \cdots (n-k+1) = n! / (n-k)!$ cách.
- Nếu **thứ tự không có ý nghĩa** (ví dụ cử k bạn đi trực): mỗi cách là một **tổ hợp**, có $C(n, k) = n! / (k! \cdot (n-k)!)$ cách.

Lê Thành Nam



Hình 1.3. *12 chỉnh hợp và 6 tổ hợp khi chọn 2 trong 4 bạn*

■ VÌ SAO C(n, k) = A(n, k) / k!?

Mỗi nhóm k vật (một tổ hợp) có thể xếp theo k! thứ tự khác nhau, mỗi thứ tự là một chỉnh hợp. Vậy số chỉnh hợp = (số tổ hợp) · k!, tức là $C(n, k) = A(n, k) / k! = n! / (k! \cdot (n - k)!)$.

■ VÍ DỤ 2

Lớp 40 bạn, cử 3 bạn đi trực: $C(40, 3) = 40 \cdot 39 \cdot 38 / 6 =$ **9 880** cách. Chọn 1 lớp trưởng, 1 lớp phó, 1 thủ quỹ (3 chức vụ khác nhau): $A(40, 3) = 40 \cdot 39 \cdot 38 =$ **59 280** cách, gấp 3! = 6 lần.

Một vài tính chất dùng rất nhiều trong sách:

■ CÔNG THỨC

$C(n, 0) = C(n, n) = 1$; $C(n, k) = C(n, n - k)$; $C(n, k) = 0$ nếu $k < 0$ hoặc $k > n$

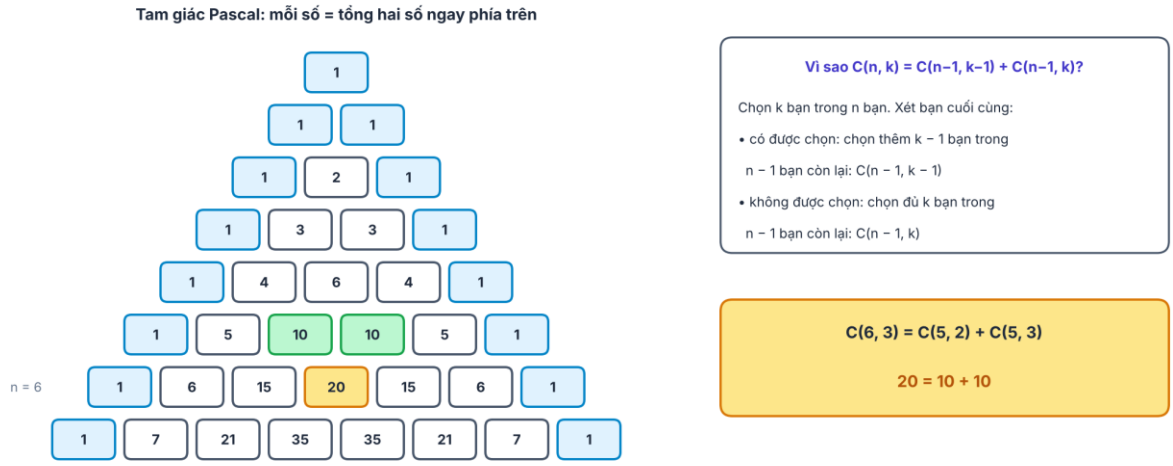
$C(n, 0) + C(n, 1) + \ldots + C(n, n) = 2^n$ (số tập con của tập n phần tử)

Lê Thành Nam

4. Tam giác Pascal

■ HẰNG ĐẲNG THỨC PASCAL

$C(n, k) = C(n - 1, k - 1) + C(n - 1, k)$ với $1 \le k \le n - 1$



Hình 1.4. Tam giác Pascal và lập luận "bạn cuối cùng có được chọn không"

■ CHỨNG MINH

Chọn k bạn trong n bạn. Chia các cách chọn làm hai loại theo bạn thứ n: có được chọn (còn phải chọn k − 1 bạn trong n − 1 bạn: C(n − 1, k − 1) cách) hoặc **không** được chọn (chọn đủ k bạn trong n − 1 bạn: C(n − 1, k) cách). Hai loại không trùng nhau, theo quy tắc cộng ta được đẳng thức.

Ưu điểm lớn của tam giác Pascal: chỉ dùng **phép cộng**, nên tính theo modulo **bất kì** đều đúng, không cần phép chia.

■ GHI NHỚ

- Cộng khi "hoặc", nhân khi "rồi".
- n! cách xếp hàng; A(n, k) = n!/(n − k)! khi có thứ tự; C(n, k) = n!/(k!(n − k)!) khi không thứ tự.
- C(n, k) = C(n − 1, k − 1) + C(n − 1, k).

PHẦN BÀI TOÁN

■ BÀI TOÁN: TOHOP

Có q câu hỏi, mỗi câu hỏi gồm hai số n, k. Hãy tính C(n, k) theo modulo $10^9 + 7$ (quy ước C(n, k) = 0 nếu k < 0 hoặc k > n).

Dữ liệu vào: Dòng đầu là q ($1 \le q \le 10^5$). Mỗi dòng sau gồm n, k ($0 \le n \le 2000$, $-1 \le k \le n + 1$).

Kết quả: Với mỗi câu hỏi, in C(n, k) mod $(10^9 + 7)$.

Lê Thành Nam

Dữ liệu vào	Kết quả	Giải thích
4 5 2 10 3 0 0 2000 1000	10 120 1 72475738	C(5, 2) = 10 (Hình 1.3 với 5 bạn). C(2000, 1000) có hơn 600 chữ số, ta chỉ in số dư.

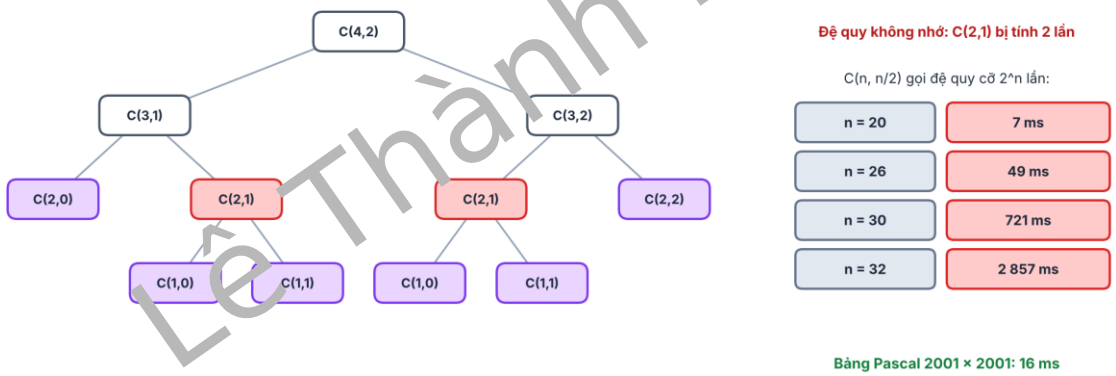
5. Hai cách ngây thơ và vì sao chúng hỏng

Cách 1: Tính giai thừa bằng số nguyên

Tính n!, k!, (n − k)! rồi chia. Với `unsigned long long`, cách này đúng tới n = 20 nhưng **sai từ n = 21**: 21! vượt 2^{64}. Đo thật: chương trình in C(21, 10) = 98 014 (đúng là 352 716) và C(30, 15) = 0. Lấy modulo trước khi chia cũng không được, vì **phép chia không "đi qua" phép lấy dư**: (a / b) mod p ≠ (a mod p) / (b mod p).

Cách 2: Đệ quy theo công thức Pascal

Viết hàm C(n, k) gọi C(n − 1, k − 1) và C(n − 1, k). Kết quả đúng, nhưng cùng một giá trị bị tính **lặp đi lặp lại**:



Hình 1.5. Cây gọi đệ quy của C(4, 2): các nút đỏ bị tính nhiều lần; thời gian tăng gấp 4 khi n tăng 2

6. Lập bảng Pascal

Tính mọi C(n, k) với n ≤ 2000 **một lần**, lưu vào bảng, rồi trả lời mỗi câu hỏi bằng một lần tra bảng.

‹› Code ý chính: Lập tam giác Pascal theo modulo

```
for (int n = 0; n <= MAXN; n++) {
    C[n][0] = C[n][n] = 1;
    for (int k = 1; k < n; k++) C[n][k] = (C[n - 1][k - 1] + C[n - 1][k]) % MOD;
}
```

▣ TOHOP.CPP: Tam giác Pascal, O(N²) tính sẵn, O(1) mỗi câu hỏi

```
#include <bits/stdc++.h>
using namespace std;
const int MOD = 1000000007;
const int MAXN = 2000;
```

Lê Thành Nam

```cpp

int C[MAXN + 1][MAXN + 1];  // C[n][k] = tổ hợp chập k của n, theo modulo

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    if (fopen("TOHOP.INP", "r")) {
        freopen("TOHOP.INP", "r", stdin);
        freopen("TOHOP.OUT", "w", stdout);
    }
    // tam giác Pascal: C(n, k) = C(n - 1, k - 1) + C(n - 1, k)
    for (int n = 0; n <= MAXN; n++) {
        C[n][0] = C[n][n] = 1;
        for (int k = 1; k < n; k++)
            C[n][k] = (C[n - 1][k - 1] + C[n - 1][k]) % MOD;
    }
    int q;
    cin >> q;
    while (q--) {
        int n, k;
        cin >> n >> k;
        cout << (k < 0 || k > n ? 0 : C[n][k]) << '\n';
    }
    return 0;
}
```

■ ĐẾM SỐ PHÉP TÍNH

- Giai thừa số nguyên: nhanh nhưng **sai** từ n = 21.
- Đệ quy không nhớ: đúng nhưng O(2ⁿ); n = 30 mất 721 ms, n = 32 mất 2,9 giây.
- Bảng Pascal: O(N²) = 2·10⁶ phép cộng tính sẵn; đo thật **16 ms**; bộ nhớ 2001 × 2001 số `int` = 16 MB.
- Giới hạn của bảng Pascal: N = 10^6 cần 10^{12} ô. Bài 3 sẽ giải quyết bằng bảng giai thừa và nghịch đảo modulo.

■ BẪY THƯỜNG GẶP

- **Nhầm tổ hợp với chỉnh hợp:** đọc kĩ đề xem thứ tự có ý nghĩa không.
- **Quên quy ước C(n, k) = 0** khi k < 0 hoặc k > n: truy cập ngoài mảng.
- **Cộng hai số rồi mới lấy modulo:** hai số < $10^9 + 7$ cộng lại vẫn vừa `int` (< 2^{31}), nhưng **nhân** hai số như vậy thì phải dùng `long long`.
- **Chia rồi lấy modulo:** sai; phải dùng nghịch đảo (Bài 3) hoặc Pascal.

Lê Thành Nam

Luyện tập

Bài	Tên	Gợi ý liên hệ
1.1	CHỌN ĐỘI	Chọn k người rồi chọn đội trưởng: quy tắc nhân k · C(n, k).
1.2	PASCAL MODULO	C(n, k) theo modulo M bất kì: chỉ Pascal là đúng.
1.3	XẾP CHỮ	Số cách xếp các chữ cái có lặp và thứ hạng của một xâu.

Tự kiểm tra nhanh:

1. Có bao nhiêu số tự nhiên có 3 chữ số khác nhau? *(Đáp số: 9 · 9 · 8 = 648: chữ số đầu khác 0.)*
2. Tính C(7, 3) bằng tam giác Pascal. *(Đáp số: 35.)*
3. Có bao nhiêu cách chọn 2 bạn nam và 1 bạn nữ từ 5 nam, 4 nữ? *(Đáp số: C(5, 2) · C(4, 1) = 40.)*

Lê Thành Nam